

Running WiFiCatcher without a fully-root process

Design proposal · privilege separation · A+ (validated sudo wrapper) vs C (privileged helper daemon)

Status: proposal / on paper. No code changed yet. **Goal of this doc:** compare two ways to stop running the whole app as root — **A+** (validated `sudo` wrapper) and **C** (privileged helper daemon) — and pick a path that keeps the operator's setup burden and OS changes minimal.

1. Goals and constraints

1. **Never run the whole app as root.** The web server, CSV parsing, file uploads and graph code must run unprivileged. A bug there must not be a root bug.
2. **Handle the genuinely-privileged features cleanly,** not by elevating everything.
3. **Minimal operator effort / minimal OS changes.** Ideally the user does not have to edit `sudoers`, install a systemd unit, or `setcap` binaries just to run the tool.
4. **Keep the beginner path.** `sudo .venv/bin/python -m WiFiCatcher` must keep working exactly as today (backward compatible).

2. What actually needs root

WiFiCatcher never opens raw sockets in Python — every privileged action is an external binary spawned via `subprocess`. Confirmed inventory:

Feature	Command(s)	Needs root?
Monitor mode on/off	<code>airmon-ng check kill</code> / <code>start</code> / <code>stop</code>	yes (netlink / iface)
Live capture	<code>airodump-ng ...</code>	yes (radio)
Deauth	<code>aireplay-ng --deauth ...</code>	yes (frame injection)
EAP enumeration	<code>EAP_buster.sh</code> → <code>wpa_supplicant</code>	yes (associates, takes iface)
Restore networking on exit	<code>systemctl restart NetworkManager</code> / <code>service ...</code>	yes
Cert extraction	<code>tshark -r <cap></code> / <code>pcapFilter.sh</code>	no — reads a file
Handshake / WPS detection	<code>tshark -r <cap></code>	no — reads a file
Interface listing	<code>/sys/class/net</code> (sysfs)	no
Web UI, parsing, upload, graph	pure Python	no

So exactly **five** command families need root. Everything that forms the real attack surface (an HTTP server that accepts file uploads and shells out) does not. That is what makes privilege reduction worthwhile and feasible.

The single gate today is `operations/base.offensive_available()` → `geteuid()==0`.

3. Threat model

We defend against a **bug in the unprivileged surface** (web server, upload handler, parser) being leveraged into code execution. Today that lands as root. The objective is that such a compromise yields, at worst, the app user's privileges plus whatever narrow, validated radio actions we deliberately expose.

We do **not** try to defend against a malicious local root, nor against physical access. We also accept that any binary we run as root (aircrack-ng suite) could in principle be abused if its own argument surface is left wide open — hence the emphasis below on not passing attacker-influenced argv straight through.

4. Design A+ — unprivileged app, one validated `sudo` wrapper

Run the app as a normal user. Route the five privileged operations through a single small **wrapper program** that is the only thing `sudo` is allowed to run. The wrapper validates the operation and its parameters, then builds the argv itself.

```
app (user) → sudo -n wc-privhelper <op> --json '{...}' → wrapper (root)
                                                    validates, runs tool
```

`sudoers` grants exactly **one** command, not the raw tools:

```
# /etc/sudoers.d/wificatcher
%wificatcher ALL=(root) NOPASSWD: /opt/wificatcher/wc-privhelper
```

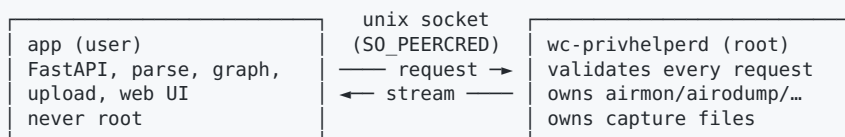
Why the wrapper instead of listing the raw binaries: if `sudoers` allowed `aireplay-ng` with any argv, a compromised app could run it as root with arbitrary flags. The wrapper closes that — it accepts only `{op: "deauth", iface, bssid, client?, count}`, validates each field (`iface` against `/sys/class/net`, MACs via `normalize_mac`, `count` clamped), and constructs the argv. The app can never inject arbitrary arguments.

Pros: small; each op is a short-lived `sudo` call; no long-running daemon. **Cons:** needs a persistent `sudoers` file (a permanent OS change, however small); live capture is a continuous `airodump-ng` process, awkward to model as short-lived wrapper calls; and capture files are written by root — the wrapper must hand them back readable.

A+ is a good fit for the one-shot operations (deauth, monitor toggle, network restart, EAP enumeration). It is a poor fit for the streaming one (live capture), which is exactly where design C shines.

5. Design C — privileged helper daemon

A tiny root daemon owns all radio work and exposes a **fixed, validated** API over a unix-domain socket. The app runs fully unprivileged and speaks to it.



Key properties:

- **Validated argv, no pass-through.** Same principle as A+: the helper builds every command line from validated fields. A compromised app cannot inject arguments or run other binaries.
- **Clean capture streaming.** The helper runs `airodump-ng`, owns its rotating CSV/pcap, tails it and **streams rows over the socket**. The app parses them as it does now, but never touches root-owned files. This removes A+'s file permission caveat entirely.
- **Socket authentication.** Socket mode `0660`, owner `root:wificatcher`; the helper additionally checks the peer uid via `SO_PEERCREd` and rejects anyone who is not the expected app user. No TCP, no network exposure.
- **Robust cleanup.** Because the helper owns the radio state, it can restore managed mode and restart NetworkManager **on client disconnect**, even if the app crashes — more robust than today, where cleanup is tied to the app process.

Cons: more code (IPC framing, serialization, error propagation, lifecycle, stream backpressure); two things to launch.

6. Launch strategy — the part that decides the OS footprint

This is where constraint #3 is won or lost. The helper can be started three ways, in increasing permanence:

Mode	What the user installs	Prompts	Footprint
On-demand (default)	nothing	one <code>sudo</code> password per session	zero permanent OS change
sudoers NOPASSWD	one <code>sudoers.d</code> line	none	one small file
systemd unit	one <code>.service</code> (+ socket)	none	service, robust, production

On-demand mode is the recommended default and directly serves "minimal OS changes": the app runs unprivileged; the first time a root feature is needed (or at startup if you know you want live capture), the app spawns **one** helper:

```
# conceptually
subprocess.Popen(["sudo", helper_path, "--socket", sock, "--peer-uid", str(os.getuid())])
```

- If a `sudoers` line exists → `sudo -n` runs it with no prompt.
- If not → the user gets **one** password prompt for the session; the helper then persists until the app exits, and dies with it. **No sudoers, no systemd, nothing left on the system.**

This gives C's security model (validated API, process boundary, clean streaming, robust cleanup) with A+'s minimal footprint — and it beats A+ on footprint, because A+ requires a

persistent `sudoers` entry while on-demand C requires nothing but a one-time password prompt.

Power users who dislike the per-session prompt opt into the `sudoers` line; production deployments use the systemd unit. Same helper, three launch modes.

7. Protocol sketch (design C)

Length-prefixed JSON messages over the socket. Fixed operation set; every field validated before anything runs.

```
→ {"op": "monitor.start", "iface": "wlan0"}
← {"ok": true, "monitor_iface": "wlan0mon"}

→ {"op": "capture.start", "iface": "wlan0mon", "channel": 6, "bssid": null, ...}
← {"ok": true, "capture_id": "c1"}
← {"stream": "c1", "csv": "<one airodump CSV snapshot>"} # repeated ~1/s
← {"stream": "c1", "csv": "..."}

→ {"op": "capture.stop", "capture_id": "c1"}
← {"ok": true, "saved_path": "/home/user/captures/..."} # chowned to app user

→ {"op": "deauth", "iface": "wlan0mon", "bssid": "AA:...", "client": null, "count": 5}
← {"ok": true, "returncode": 0}

→ {"op": "eap.enumerate", "iface": "wlan0", "essid": "corp", "identity": "DOM\\u"}
← {"ok": true, "methods": [...]}

→ {"op": "network.restart"}
← {"ok": true}
```

Validation rules reuse existing code: `iface` must appear in `/sys/class/net` and be wireless; MACs via `models.normalize_mac`; `count` clamped to `MAX_COUNT` (64); `channel` an int in range; `essid` / `identity` via the existing regexes. Unknown `op`, bad field, or a peer-uid mismatch → hard reject, logged.

Cert extraction, handshake and WPS detection stay in the **app** (they only read files); for a live capture, the helper hands back a readable copy or path so the app can run `tshark` unprivileged.

8. Backward compatibility and the code gate

- If the process is already root (`sudo .venv/bin/python ...`), the helper is not needed: run the tools in-process exactly as today. `elevate()` / the client short-circuits. **Nothing changes for existing users.**
- `offensive_available()` changes from "am I root?" to "am I root **or** is a helper reachable/launchable?".
- The privileged bodies in `operations/` and `capture/` are already isolated; they become the helper's implementation almost as-is. The app side gets thin client stubs.

9. Recommendation

Adopt **design C with on-demand launch as the default**, because it is the only option that satisfies all three constraints at once:

- never runs the app as root ✓
- handles the five privileged features through a small, validated, auditable surface ✓

- **zero permanent OS modification** in the default mode (one password prompt per session), with opt-in `sudoers` /systemd for those who want no prompt ✓

A+ remains a valid, lighter milestone: if we want value sooner, we can ship the validated `wc-privhelper` wrapper first (it is literally the helper's request handler without the socket/daemon/streaming) and drive the one-shot operations through `sudo` immediately, then grow it into the on-demand daemon for live capture. A+ and C share the same validation core — A+ is C without the socket.

10. Migration in phases

1. **Extract & validate.** Pull the privileged command builders into a single module with strict per-op validation (shared by every later phase). No behavior change yet.
2. **A+ wrapper.** Ship `wc-privhelper <op>` ; route the four one-shot ops through `sudo -n` (fallback: interactive `sudo`). Document the optional one-line `sudoers` .
3. **C daemon + streaming.** Turn the wrapper into a socket daemon; move live capture to streamed CSV; add `SO_PEERCRED` auth and on-disconnect cleanup.
4. **Deployment.** Provide an optional systemd unit for production; keep on-demand as the default and `sudo` -all as the legacy fallback.

11. Open questions / risks

- `airmon-ng` is a shell script, so `setcap` is out — one more reason to gate through the helper rather than capabilities.
- **Helper validation bugs are still root bugs**, but on a tiny fixed surface — the tradeoff we are explicitly buying.
- **On-demand prompt UX:** where to prompt (terminal vs. a UI hint) when the app is started from a browser context; likely terminal-only, mirroring today.
- **Saved-capture ownership:** the helper must `chown` kept captures to the app user (design C) so the user owns their data.
- **Distro paths** for `systemctl` / `service` /tool locations must be resolved, not hardcoded.